Papelería Matrix: estructura del proyecto

Cómo está organizado el repositorio hoy, cómo viaja una petición por el backend, qué tablas ya tienen su clase nueva y cómo se configura y se despliega todo.

Vista general del repositorio

```
Sistema-papeleria-matrix/
├── 01-requisitos/ a 08-contexto/    Documentación del proyecto
│                                    (06-documentacion guarda este paquete)
├── database/scripts/                Scripts SQL
│   ├── 01-crear-base-de-datos/
│   ├── 02-crear-tablas-base-de-datos/   15 tablas
│   └── 03-crear-relaciones-fk/          14 claves foráneas
├── backend/                         API en Spring Boot 4.1 y Java 21 (Maven)
│   ├── pom.xml
│   └── src/
│       ├── main/java/com/papeleriamatrix/   Código, por paquetes
│       ├── main/resources/                  application*.yml y datos de ejemplo
│       └── test/java/                       24 pruebas
├── frontend/                        Página pública (HTML, CSS y JavaScript)
│   ├── index.html, views/login.html
│   └── css/, js/, assets/
├── postman/                         Colección de la API y sus entornos
├── .github/workflows/ci.yml         Integración continua
├── Dockerfile, .dockerignore        Imagen de la API
├── render.yaml                      Configuración de Render
├── CONTRIBUTING.md                  Cómo trabajamos
└── README.md
```

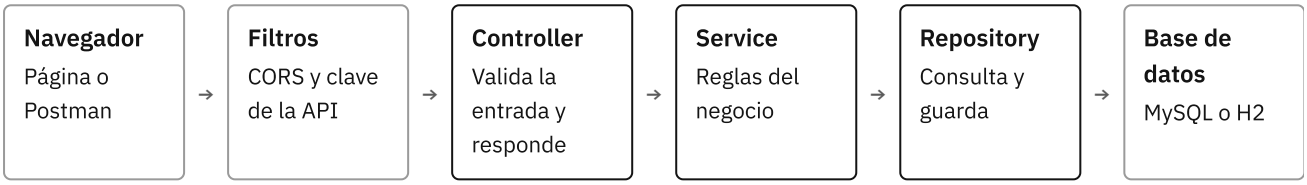
El backend por dentro

El código está en `backend/src/main/java/com/papeleriamatrix`, dividido en paquetes. Cada uno tiene una sola responsabilidad.

PAQUETE	RESPONSABILIDAD	CONTENIDO HOY
`controller`	Recibe las peticiones HTTP y devuelve respuestas.	`ServicioController`, `ProductoPapeleriaController`, `RopaController`, `EstadoController`, `InicioController`
`service`	Reglas del negocio.	`ServicioService`, `ProductoPapeleriaService`, `RopaService`
`repository`	Acceso a la base de datos. Reemplaza a los DAO.	`ServicioRepository`, `ProductoPapeleriaRepository`, `RopaRepository`
`model`	Una clase por tabla.	3 entidades JPA nuevas y 12 clases antiguas (ver el mapa de tablas)

`dto`	Lo que entra y sale en JSON.	`ServicioRequest`, `ServicioResponse`, `ProductoResponse`, `RopaResponse`, `EstadoRespuesta`
`exception`	Convierte los errores en respuestas claras.	`ManejadorDeErrores`, `RecursoNoEncontradoException`, `RecursoDuplicadoException`
`security`	La clave de la API para escribir.	`ClaveApiFilter`
`config`	Configuración.	`OpenApiConfig` (Swagger), `CorsConfig`, `ConexionBD` (antigua)
`dao`	Los DAO con JDBC de la versión anterior.	12 clases. 3 tienen código y 9 están vacías. Se van reemplazando por repositorios.
`common`	Utilidades.	`Utils`

Esquema de capas



Cuando algo falla en cualquier capa, `ManejadorDeErrores` lo convierte en un JSON con un título y un detalle claros, y deja el error técnico solo en el registro del servidor (RNF-05).

Ejemplo: crear un servicio

1. Llega `POST /api/servicios` con un JSON y la cabecera `X-API-Key`.
2. El filtro de CORS deja pasar el origen permitido. El filtro de la clave comprueba la cabecera: si falta, responde 401.
3. `ServicioController` convierte el JSON en un `ServicioRequest` y lo valida: nombre no vacío, precio mayor que cero. Si falla, 400 con los campos.
4. `ServicioService` comprueba que el código no exista: si existe, 409.
5. `ServicioRepository` guarda la entidad `Servicio` en la tabla `servicio`.
6. El controlador responde 201 con un `ServicioResponse` y la dirección del nuevo servicio en la cabecera `Location`.

Mapa de tablas y clases

La base tiene 15 tablas. Tres ya tienen su entidad JPA con repositorio, servicio y controlador. Las otras doce conservan su clase y su DAO antiguos, a la espera de pasar al mismo patrón.

TABLA	CLASE HOY	ESTADO	TAREA
`servicio`	`Servicio`	● `JPA y API completa`	Hecha
`producto_de_papeleria`	`ProductoPapeleria`	◐ `JPA, solo consulta`	T1
`ropa`	`Ropa`	◐ `JPA, solo consulta`	T2
`usuario`	`Usuario` y `UsuarioDAO` (con código)	○ `Clase antigua`	T3

`administrador`	`Administrador` y `AdministradorDAO` (con código)	○ **Clase antigua**	T3
`informacion_del_negocio`	`Informacion_del_negocio` y DAO vacío	○ **Clase antigua**	T4
`operador_de_cibercafe`	`Operador_de_cibercafe` y DAO (con código)	○ **Clase antigua**	T5
`clave_de_permiso`	`Clave_de_permiso` y DAO vacío	○ **Clase antigua**	T5
`factura_del_negocio`	`Factura_del_negocio` y DAO vacío	○ **Clase antigua**	T6
`detalle_de_factura`	`Detalle_de_factura` y DAO vacío	○ **Clase antigua**	T6, T7
`ingreso_del_negocio`	`Ingreso_del_negocio` y DAO vacío	○ **Clase antigua**	T8
`historial_ingreso_del_negocio`	`Historial_ingreso_del_negocio` y DAO vacío	○ **Clase antigua**	T8
`estadistica_del_negocio`	`Estadistica_del_negocio` y DAO vacío	○ **Clase antigua**	T8
`historial_estadistica_del_negocio`	`Historial_estadistica_del_negocio` y DAO vacío	○ **Clase antigua**	T8
`reporte_del_negocio`	`Reporte_del_negocio` y DAO vacío	○ **Clase antigua**	T8

Las relaciones entre tablas siguen en `database/scripts/03-crear-relaciones-fk`. Las dependencias principales son: `administrador` y `operador_de_cibercafe` cuelgan de `usuario`; `informacion_del_negocio` y `reporte_del_negocio` cuelgan de `administrador`; `clave_de_permiso` y `factura_del_negocio` cuelgan de `operador_de_cibercafe`; y `detalle_de_factura` apunta a la factura y, de forma opcional, a un servicio, un producto o una prenda.

Rutas de la API

RUTA	QUÉ HACE	CLAVE
`GET /`	Lleva a la documentación.	No
`GET /api/estado`	Dice si la API está en línea y en qué modo corre.	No
`GET /actuator/health`	La usan Render y el CI para saber si la API está viva.	No
`GET /api/servicios`, `GET /api/servicios/{id}`	Lista y consulta servicios.	No
`POST /api/servicios`	Crea un servicio. Responde 201.	Sí
`PUT /api/servicios/{id}`	Edita un servicio.	Sí
`DELETE /api/servicios/{id}`	Elimina un servicio. Responde 204.	Sí
`GET /api/productos`, `GET /api/productos/{id}`	Catálogo de papelería, sin precio de compra ni proveedor.	No
`GET /api/ropa`, `GET /api/ropa/{id}`	Catálogo de ropa.	No

`/swagger-ui.html` , `/v3/api-docs`	La documentación de la API, en pantalla y en formato OpenAPI.	No

Los errores salen en JSON con título y detalle: 400 si los datos son inválidos, 401 si falta la clave, 404 si no existe, 405 si el método no se admite y 409 si el código ya está en uso. La clave provisional es un paso intermedio: el inicio de sesión con roles la reemplazará.

Configuración

La API lee su configuración de `application.yml` , de un perfil (`application-local.yml` o `application-demo.yml`) y de variables de entorno.

VARIABLE	QUÉ CONTROLA	VALOR POR DEFECTO
`SPRING_PROFILES_ACTIVE`	El modo: `local` o `demo` .	`local` . En el contenedor de Docker, `demo` .
`PORT`	El puerto de la API.	`8080`
`DB_URL` , `DB_USER` , `DB_PASSWORD`	La conexión a MySQL en modo local.	`jdbc:mysql://localhost:3306/papeleria_matrix` , usuario `root` , sin contraseña
`API_KEY`	La clave que se exige para crear, editar y borrar.	`clave-de-prueba` . En Render la genera el propio servicio.
`CORS_ORIGENES`	Las direcciones desde las que el frontend puede consultar la API.	`http://localhost:5500,http://127.0.0.1:5500`

MODO	BASE DE DATOS	TABLAS	PARA QUÉ SIRVE
local	MySQL de XAMPP	Las crean los scripts SQL. Al arrancar, `ddl-auto: validate` compara las entidades con ellas y avisa de cualquier diferencia de nombre.	El sistema real.
demo	H2 en memoria	Las crea Hibernate desde las entidades y se cargan los datos de `demo/datos-demo.sql` .	Render, las pruebas y las demostraciones. Los datos se pierden al reiniciar.

El frontend

ARCHIVO	PARA QUÉ SIRVE
`index.html`	La página principal: inicio, nosotros, horarios, servicios, papelería, ropa y contacto.
`views/login.html`	La pantalla de inicio de sesión. Todavía no da acceso.
`css/bases/`	Fuentes, variables de la marca y reset.
`css/vistas/`	Los estilos de cada vista.
`js/config.js`	La dirección de la API. En el computador local usa el puerto 8080 y publicada usa Render.
`js/datos-respaldo.js`	El catálogo que se muestra si la API no responde.

`js/main.js`	Menú, modales, catálogos, buscadores y aviso de privacidad.

La página pide los datos a la API desde el navegador. Por eso la API necesita CORS. Mientras no exista la API de información del negocio, los horarios y el contacto están escritos en el HTML.

Despliegue y CI

PIEZA	QUÉ HACE
`Dockerfile`	Dos etapas: Maven con Java 21 compila el jar, y una imagen con solo el JRE 21 lo ejecuta, con parámetros de memoria pensados para el plan gratuito de Render (512 MB). Arranca en modo demo.
`render.yaml`	Describe el servicio en Render: Docker, plan gratuito, comprobación de salud en `/actuator/health`, la variable `API_KEY` generada por Render y despliegue solo cuando pasan las revisiones de GitHub (`checksPass`).
`.github/workflows/ci.yml`	Dos trabajos en cada PR y en cada commit a `main`: «Backend (compilar y probar)» y «Docker (construir la imagen)».

El plan gratuito de Render duerme el servicio tras 15 minutos sin uso y tarda cerca de un minuto en despertar.

Convenciones

- **Idioma.** Texto visible, nombres de clases, métodos y variables en español. Los nombres técnicos del lenguaje y de las librerías se dejan en inglés.
- **Tablas.** En minúscula y con guion bajo. Las columnas conservan sus nombres del script, como `Id_servicio`.
- **Estilo del código Java.** Tabulaciones y llaves en su propia línea.
- **Estilo del frontend.** Cuatro espacios y nombres de clases CSS con la forma BEM, como `card__title`.
- **Ramas.** `Feat/`, `Fix/`, `Refactor/` y `Docs/`, seguidos de un nombre corto.
- **Commits.** Con prefijo: `feat:`, `fix:`, `refactor:`, `docs:`, `ci:`, `chore:`.
- **Secretos.** Ninguna contraseña ni clave real en el repositorio. Todo va por variables de entorno.

Decisión por registrar

> **D-01. Backend con Spring Boot y JPA.** La descripción de tecnologías del proyecto establece Java con JDBC y DAO, y RNF-17 pide que cualquier tecnología fuera de ese rango la apruebe el líder del proyecto. El backend actual usa Spring Boot 4.1, Spring Data JPA, Maven, Docker y GitHub Actions. Los motivos: Spring Boot aporta servidor HTTP, validación y pruebas que antes no existían, y los repositorios hacen el mismo trabajo que los DAO con menos código. Falta registrar la aprobación y actualizar el documento *descripción de tecnologías*.